



数据结构与算法 I

实验手册

组织：人大信息学院
时间：2024 年 9 月
版本：20240930

前言

所有实验需按照教学系统指定的日期提交。晚交的分数调整规则如下：假定 x 为作业原始得分， d 为晚交天数，则新分数为 $x \cdot e^{-\frac{d}{7} \times 0.1}$ 。

所有实验（含平时作业）均要求每位同学独立完成。对于提交的源代码，我们将作代码查重。若查重软件提示重复超过 40%，将人工核查。若被判定为“过度引用”，不管引用方还是被引用方，本次实验均将计 0 分，并视情节严重程度上报学院和学校教务。

除非另外说明，本实验手册默认的硬件设备为个人计算机，软件环境为 C/C++ 开发环境。

注意：禁止使用 C++ 中的 STL！

每个实验还需要大家完成实验报告 and 用户使用手册。大家参考本实验书第一章的模板完成，**明确要点即可，不鼓励大家卷文字内容，过多的文字内容也不会加分。**最终需提交 pdf 文件。

本手册使用了 **Elegant $\LaTeX$** 模板编写。

本手册为内部讲义，禁止外传。

目录

第 1 章 实验报告和用户使用手册模板	1
1.1 实验报告	1
1.2 用户使用手册	1
第 2 章 基础实验	2
2.1 目的	2
2.2 任务	2
2.3 测试样例	2
2.4 评分标准	3
第 3 章 实验一	5
3.1 目的	5
3.2 任务	5
3.3 测试样例	5
3.4 评分标准	6
第 4 章 实验二	7
4.1 目的	7
4.2 任务	7
4.3 测试样例	7
4.4 评分标准	8
第 5 章 实验三	9
5.1 目的	9
5.2 任务	9
5.3 测试样例	9
5.4 评分标准	11
第 6 章 实验四	12
6.1 目的	12

6.2 任务	12
6.3 测试样例	12
6.4 评分标准	13
第 7 章实验五	14
7.1 目的	14
7.2 任务	14
第 8 章实验六	15
8.1 目的	15
8.2 任务	15
8.3 测试样例	16
8.4 评分标准	17

第 1 章 实验报告和用户使用手册说明

请同学们在`overleaf`平台使用 `LaTeX` 语言按照模板完成实验报告和用户使用手册，最终以 pdf 格式提交，分别命名为【姓名_学号_实验报告.pdf】和【姓名_学号_用户使用手册.pdf】。提交文件夹需包含实验报告、用户使用手册、程序代码、可执行程序及输入输出文件（若有）。

1.1 实验报告

实验报告是把实验方案、过程和结果等用文字、图表等形式记录下来的书面材料。实验报告需包含下列章节¹：

1. 问题分析：这个问题你是怎么抽象成数据结构问题的？编程时需要完成哪些内容？
2. 数据结构设计与实现
 - ADT 设计：需包含数据对象、数据关系、基本操作。
 - ADT 的物理实现：需要列出构建数据对象的代码片段。（基本操作只需列出函数声明即可。）
3. 算法设计与实现
 - 算法设计：简要说明解决这个问题所需要的算法思想与步骤。分析该算法的时间/空间复杂度分析。
 - 算法实现：列出完成具体算法所需的关键代码片段，并简单介绍。
4. 实验环境
 - 硬件环境：CPU 型号、内存
 - 软件环境：操作系统、编程语言、IDE 及版本
5. 实验结果与分析

实验报告参考模板链接：<https://cn.overleaf.com/read/dbqckgjyqsgw#7610e8>

1.2 用户使用手册

用户使用手册描述了用户（比如其他同学、助教）如何使用你所开发的软件（如.exe 可执行程序）。因此，该手册就是一个简要的软件使用说明书。

需描述完成该实验所需的操作步骤与输入内容，并列出关键步骤后的程序输出截图。

用户使用手册参考模板链接：<https://cn.overleaf.com/read/mscnfprwmpvh#6a1e2a>

¹前序实验报告中已经给出的实现直接引用相应实验报告即可，无需重复说明

第 2 章 文件读写与向量

2.1 目的

1. 掌握 C/C++ 读写文本/二进制文件
2. 掌握向量的实现与基本操作

2.2 任务

使用包含数组的结构体或 class 类模拟向量的实现，实现增删改查等功能。共有 9 种指令：

1. `load [filename.ext]`：从对应文本/二进制文件中读取数据，来初始化向量，并根据后续指定命令对向量进行更新、打印和保存。
2. `size`：打印向量中元素个数。
3. `show`：在一行内按序打印向量中所有数值。
4. `push_back [x]`：在向量末尾加入 [x] 数值，然后在一行内按序打印向量中所有数值。
5. `pop_back`：删除向量在末尾处的数值，然后在一行内按序打印向量中所有数值。
6. `insert [p] [x]`：将数值 [x] 加入到向量的第 [p] 个位置，然后在一行内按序打印向量中所有数值。
7. `erase [p]`：删除向量中第 [p] 个位置的数值，然后在一行内按序打印向量中所有数值。
8. `update [p] [x]`：更新向量中第 [p] 个位置的数值为 [x]，然后在一行内按序打印向量中所有数值。
9. `save [filename.ext]`：将向量按序保存为文件，根据后缀是 `txt` 还是 `bin` 自动决定存储方式。不支持的格式要报错。

2.3 测试样例

- 输入：`load [your_file]`
 - 从文本文件 `exp1_example_input.txt` 中读取数据。
文件内容：
5(向量中元素个数)

3 65 678 32 6(向量中的每个元素, 默认为 int 类型)

输出: Loaded successfully from [your_path]/exp1_example_input.txt (text)!

- 从二进制文件 exp1_example_input.bin 中读取数据

文件内容:

5(向量中元素个数)

3 65 678 32 6(向量中的每个元素, 默认为 int 类型)

输出: Loaded successfully from [your_path]/exp1_example_input.bin (binary)!

- 向量上的几种操作

- 输入: show

输出: 3 65 678 32 6

- 输入: push_back 23

输出: 3 65 678 32 6 23

- 输入: pop_back

输出: 3 65 678 32 6

- 输入: insert 2 123

输出: 3 123 65 678 32 6

- 输入: erase 3

输出: 3 123 678 32 6

- 输入: update 2 120

输出: 3 120 678 32 6

- 输入: size

输出: 5

- 输入: save exp1_example_output.txt

输出: Saved successfully to [your_path]/exp1_example_output.txt (text)!

- 输入: save exp1_example_output.bin

输出: Saved successfully to [your_path]/exp1_example_output.bin (binary)!

- 输入: exit

退出程序

2.4 评分标准

- 助教现场检查: 40 分

- 向助教清楚解释设计要点、关键代码 20 分
- 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分

第3章 数据的读取和查询

3.1 目的

1. 理解数据结构中的 ADT 等基本概念
2. 掌握使用 C/C++ 读取文件的功能
3. 掌握基本的数据查询功能

3.2 任务

设计合适的数据结构，从指定文件（flight-data.csv）读取航班数据，并根据指定条件查询相应航班。

1. 观察航班数据，设计合适的数据结构，读取航班数据。（针对日期的读取操作推荐使用 ctime 库中的 tm 时间结构体以及 std::get_time 函数或 asctime 函数。）
2. 完成数据读取后，统计所有数据中起飞最早、最晚的时间；花费时间最短、最长的航班；价格最贵、最便宜的航班。请给出相应统计结果。
3. 完成数据读取后，查询两个机场之间是否有直达航班。如果有，请给出所有航班的 ID 信息；如果没有，请输出相应提示信息。
4. 从直达航班中进一步找出满足下列条件的航班 ID
 - (a). 费用最少
 - (b). 用时最少
 - (c). 降落时间是起飞时间的第二天

 **笔记** 判断两个日期是否是相邻日期时，应注意跨月或跨年的情况。

3.3 测试样例

- 读取文件后，输出统计数据：

Statistics of flights:

Earliest departure time: Fri May 5 00:30:00 2017, Flight ID: 1417

Latest departure time: Mon May 8 22:30:00 2017, Flight ID: 405

Shortest duration: 0.83 hours, Flight ID: 544

Longest duration: 15.5 hours, Flight ID: 1454

Cheapest flight: ¥181, Flight ID: 545

Most expensive flight: ¥10281, Flight ID: 1454

- 查询机场 48 到机场 50 的直达航班

输入: 48 50

输出: 1 (cheapest), 2, 3, 5, 83 (shortest duration), 84 (shortest duration), 85 (shortest duration)

- 查询机场 49 到机场 1 的直达航班

输入: 49 1

输出: 343 (cheapest, shortest duration, +1 day), 344 (shortest duration, +1 day)

- 查询机场 49 到机场 50 的直达航班

输入: 49 50

输出: No direct flight.

- 退出程序

输入: 0 0

退出程序

3.4 评分标准

- 助教现场检查: 40 分
 - 向助教清楚解释设计要点、关键代码 20 分
 - 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分

第4章 数据的增加、删除与修改

4.1 目的

1. 掌握顺序表/链表的基本知识和使用技术
2. 培养对问题抽象和建模的能力
3. 培养设计和开发简单软件的能力

4.2 任务

在实验1的基础上，设计合适的数据结构，实现实时增加、删除和修改航线信息的功能。

1. 实时增加、删除和修改单条航线的信息。对于不合法的增加、删除和修改，需给出相应的错误提示。
2. 增加、删除或修改航线信息后，从最新直达航班信息中找出满足下列条件的航班，并作相应标记 (cheapest, shortest duration, +1 day)
 - (a). 费用最少
 - (b). 用时最少
 - (c). 降落时间是起飞时间的第二天
3. 思考题：如何设计更加用户友好的输入模式？测试样例仅供参考

 **笔记** 航线更新应及时生效，程序重新启动后也应读取更新后的航线信息。

4.3 测试样例

- 删除 ID 为 19 的航班，并查询机场 36 到机场 4 的直达航班：

输入：show 36 4

输出：19 (cheapest, shortest duration, +1 day)

输入：delete 19

输出：Delete successfully.

输入：show 36 4

输出：No direct flight.

- 承接上一步操作，增加 ID 为 2347 的航班：

输入：`insert 2347,5/6/2017,362,36,4,5/7/2017 18:45,5/6/2017 10:25,25,4,6571`

输出：`Input format error, please input again.`

输入：`insert 2347,5/6/2017,intl,362,36,4,5/7/2017 18:45,5/6/2017 10:25,25,4,6571`

输出：`Departure time is later than arrival time, please input again.`

输入：`insert 2347,5/6/2017,intl,362,36,4,5/6/2017 18:45,5/7/2017 10:25,25,4,6571`

输出：`Insert successfully.`

- 承接上一步操作，增加 ID 为 2348 的航班，并查询机场 36 到机场 4 的直达航班：

输入：`insert 2348,5/6/2017,intl,362,36,4,5/6/2017 12:45,5/7/2017 1:25,24,4,7550`

输出：`Insert successfully.`

输入：`show 36 4`

输出：`2347 (cheapest, +1 day), 2348 (shortest duration, +1 day)`

- 承接上一步操作，修改 ID 为 2347 的航班，并查询机场 36 到机场 4 的直达航班：

输入：`update 2347,5/6/2017,intl,362,36,4,5/6/2017 22:10,5/7/2017 10:25,25,4,8571`

输出：`update successfully.`

输入：`show 36 4`

输出：`2347 (shortest duration, +1 day), 2348 (cheapest, +1 day)`

- 承接上一步操作，修改 ID 为 19 的航班：

输入：`update 19,5/5/2017,intl,362,36,4,5/5/2017 12:45,5/6/2017 1:25,24,4,7259`

输出：`No such flight, please input again.`

4.4 评分标准

- 助教现场检查: 40 分
 - 向助教清楚解释设计要点、关键代码 20 分
 - 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分

第 5 章 最多次乘机问题

5.1 目的

1. 掌握顺序表/链表/栈的基本知识和使用技术
2. 培养借助网上资源自主学习的能力
3. 培养对问题抽象和建模的能力
4. 培养设计和开发具有一定复杂度软件的能力

5.2 任务

1. 在实验 2 的基础上，实现从文件内批量增加、删除与修改航班信息。
2. 增加暂停和恢复航班功能。比如某机场因极端天气影响，则在该机场上所有起降的航班均需暂停，相应的有恢复该机场所有航班的功能。
3. 最多次乘机问题。给定起始航班 ID，求最多能乘坐多少次航班，输出所有符合要求的航线 (航班 ID 顺序表)，要求同一机场换乘需留出 30 分钟以上（含 30 分钟）的时间。

 **笔记** 任务 3 乘机问题需考虑航班是否是暂停状态。

5.3 测试样例

- 从文件中批量增加、删除和修改数据。

输入文件内容 input.txt:

```
delete,19

insert,2347,5/6/2017,362,36,4,5/7/2017 18:45,5/6/2017 10:25,25,4,6571

insert,2347,5/6/2017,intl,362,36,4,5/7/2017 18:45,5/6/2017 10:25,25,4,6571

insert,2347,5/6/2017,intl,362,36,4,5/6/2017 18:45,5/7/2017 10:25,25,4,6571

update,2347,5/6/2017,intl,362,36,4,5/6/2017 22:10,5/7/2017 10:25,25,4,8571

update,19,5/5/2017,intl,362,36,4,5/5/2017 12:45,5/6/2017 1:25,24,4,7259
```

输出:

Successfully deleted 1 record;

Successfully inserted 1 record, failed to insert 2 records;

Successfully updated 1 record, failed to update 1 record.

- 航班暂停与恢复。输入要暂停或恢复的机场 ID，输出相应提示信息。

输入：Suspend, 50

输出：Suspend successfully.

输入：show, 50, 48

输出：No direct flight.

输入：Reactivate, 50

输出：Reactivate successfully.

输入：show, 50, 48

输出：9 (shortest duration), 10 (shortest duration), 11 (shortest duration), 12 (shortest duration),
70, 71, 72 (cheapest)

- 最多次乘机问题，输入待查询起始航班号，输出所有最长航线。

输入：Max, 300

输出：The maximum number of flights is 12, all paths are as follow:

300 -> 1830 -> 1835 -> 1843 -> 1847 -> 1601 -> 1257 -> 1046 -> 1074 -> 1407 -> 1414 -> 180

300 -> 1830 -> 1835 -> 1843 -> 1847 -> 1601 -> 1257 -> 1046 -> 1074 -> 486 -> 1022 -> 1489

300 -> 1830 -> 1835 -> 1843 -> 1847 -> 1596 -> 1257 -> 1046 -> 1074 -> 1407 -> 1414 -> 180

300 -> 1830 -> 1835 -> 1843 -> 1847 -> 1596 -> 1257 -> 1046 -> 1074 -> 486 -> 1022 -> 1489

输入：Suspend, 35

输出：Suspend successfully.

输入：Max, 300

输出：The maximum number of flights is 11, all paths are as follow:

300 -> 1461 -> 1458 -> 2077 -> 2082 -> 296 -> 288 -> 201 -> 190 -> 482 -> 1579

300 -> 1461 -> 1458 -> 2077 -> 2082 -> 296 -> 288 -> 201 -> 190 -> 482 -> 1124

300 -> 1461 -> 1458 -> 2077 -> 2082 -> 296 -> 288 -> 244 -> 1285 -> 482 -> 1579

300 -> 1461 -> 1458 -> 2077 -> 2082 -> 296 -> 288 -> 244 -> 1285 -> 482 -> 1124

300 -> 1461 -> 1513 -> 716 -> 2082 -> 296 -> 288 -> 201 -> 190 -> 482 -> 1579

300 -> 1461 -> 1513 -> 716 -> 2082 -> 296 -> 288 -> 201 -> 190 -> 482 -> 1124

300 -> 1461 -> 1513 -> 716 -> 2082 -> 296 -> 288 -> 244 -> 1285 -> 482 -> 1579

300 -> 1461 -> 1513 -> 716 -> 2082 -> 296 -> 288 -> 244 -> 1285 -> 482 -> 1124

5.4 评分标准

- 助教现场检查: 40 分
 - 向助教清楚解释设计要点、关键代码 20 分
 - 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分

第 6 章 机场近似搜索和推荐

6.1 目的

1. 掌握顺序表、链表、串和数组等结构的基本知识和使用技术
2. 培养借助网上资源自主学习的能力
3. 培养对问题抽象和建模的能力
4. 培养设计和开发较复杂软件的能力

6.2 任务

在实验 3 的基础上，增加机场名称近似搜索功能，和相同省份机场推荐功能。

设计合适的数据结构，从文件（id2name.csv）读取机场 id 与机场信息的对应关系。机场信息格式为：国家/省份/机场名称。

1. 机场名称近似搜索。对于输入的关键词，按照近似分数降序列出最相似的 5 个机场名称。
2. 相同省份机场推荐。查询时，若两个机场之间没有直达航班，且起飞机场与到达机场相同省份的机场有直达航班，则输出相应的直达航班。

 **笔记** 自行学习近似搜索算法，测试样例使用的是最长公共子串，不同算法的结果可能不同。

6.3 测试样例

- 机场名称的近似搜索。输入要搜索的机场名称，按照相似分数降序输出最相似的 5 个机场名称。

输入：Search Changshui

输出：Top 5 most similar airport names are as follow:

Kunming Changshui

Changsha Huanghua

Nanchang Changbei

Fuzhou Changle

Changchun Longjia

- 相同省份机场推荐。查询时，若无直达航班，则输出推荐航班。

输入: show,36,48

输出: No direct flight. There are 5 direct flights from airports in the same province as the arrival airport: 73 (shortest duration), 487, 488 (cheapest), 489, 490

- 相同省份机场推荐。查询时,若无直达航班,且无推荐航班,则只输出提示。

输入: show,36,26

输出: No direct flight.

6.4 评分标准

- 助教现场检查: 40 分
 - 向助教清楚解释设计要点、关键代码 20 分
 - 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分

第7章 最繁忙的机场

7.1 目的

1. 掌握图结构的基本知识，熟练使用邻接矩阵、邻接表等图的物理存储结构。
2. 培养对问题建模和抽象的能力。
3. 培养设计和使用新工具的能力。

7.2 任务

1. 统计出最繁忙的机场是哪个，并分别给出起飞的航班和降落的航班的个数。
 - (a). 不限制时间要求，给出结果。
 - (b). 给定起飞时段以及降落时段要求，给出结果。

第 8 章 乘机方案查询

8.1 目的

1. 掌握图结构的基本知识，熟练使用邻接矩阵、邻接表等图的物理存储结构。
2. 培养对问题建模和抽象的能力。
3. 培养设计和使用新工具的能力。
4. 能够撰写实验（技术）报告，培养书面沟通能力。

8.2 任务

1. 仅限直飞或 1 次中转，求任意两个机场的可连通性。输入出发机场和到达机场 ID，分别求下面两种情况下的两个机场的连通性，并给出所有可行的乘机方案（即对应的航班 ID 序列）。没有的话，输出提示信息。
 - (a). 不限定起飞时间，经过直飞或者 1 次转机能到达即可
 - (b). 给定起飞时段或者降落时段要求，经过直飞或者 1 次转机到达
2. 求任意两个机场之间的最优乘机方案。输入出发机场和到达机场 ID，不限定起飞时间和中转次数，分别给出下面条件最优的乘机方案。如有多种方案，需要全部输出。
 - (a). 最短飞行时间（含转机停留时间）
 - (b). 最低航费（机票价格）
3. 假定小明要从 A 地到 B 地出差，要求在某时间 t_s 之后登机，某时间 t_e 之前下飞机（不考虑去机场的时间），小明最多能够承受的转机次数是 N。输入为 A、B、 t_s 、 t_e 、N 的值，分别给出下面条件最优的乘机方案。如有多种方案，需要全部输出。
 - (a). 最短飞行时间（含转机停留时间）
 - (b). 最低航费（机票价格）

 **笔记** 注意不形成回路，已经走过的机场不能再返回。

同一机场换乘需留出 30 分钟以上（含 30 分钟）的时间。

8.3 测试样例

按照下面示例格式化输出，先列出 *options* 总数，然后、然后每个 *option* 先给出关键信息 (转机次数，累计时间开销，总机票价格)，最后再给出具体航线信息 (*Flight ID*)。如有换乘，请依次输出该航线的的所有换乘时间

- 不限定起飞时间，经过直飞或者 1 次转机条件下，任意两个机场的可连通性检查。

输入: Connect, 50, 25

输出: There are 140 options that meet that meet this query:

[option1]: 0 transfer, 0 days 3 hours 0 mins, 1128 dollars

502

[option2]: 0 transfer, 0 days 3 hours 0 mins, 1121 dollars

503

...

[option25]: 1 transfer, 0 days 19 hours 40 mins, 1139 dollars

226 -> 719 (0 days 16 hours 5 mins)

[option26]: 1 transfer, 1 days 19 hours 40 mins, 1352 dollars

226 -> 720 (1 days 16 hours 5 mins)

...

输入: Connect, 48, 4

输出: There are no options that meet that meet this query.

- 限定起飞时间，经过直飞或者 1 次转机条件下，任意两个机场的可连通性检查。依次输入起飞机场和到达机场 ID，起飞时间限制，降落时间段限制 (*none* 表示不需要该限制)

输入: Connect, 50, 25, 5/5/2017 12:00 - 5/5/2017 18:00, none

输出: There are 13 options that meet this query:

[option1]: 0 transfer, 0 days 3 hours 0 mins, 1252 dollars

515

...

[option4]: 1 transfer, 0 days 23 hours 45 mins, 1493 dollars

1270 -> 719 (0 days 20 hours 5 mins)

...

- 任意两个机场之间最短飞行时间的乘机方案

输入: Shortest, 50, 25

输出: There are 4 options that meet this query:

[option1]: 0 transfer, 0 days 2 hours 45 mins, 1384 dollars

730

...

- 任意两个机场之间最低航费的乘机方案

输入: Cheapest, 50, 26

输出: There are 1 options that meet this query:

[option1]: 1 transfer, 1 days 4 hours 50 mins, 1115 dollars

779 -> 771 (1 days 1 hours 0 mins)

输入: Cheapest, 2, 45

输出: There are 1 options that meet this query:

[option1]: 2 transfers, 1 days 23 hours 50 mins, 8212 dollars

591 -> 873 -> 2306 (1 days 2 hours 25 mins, 0 days 5 hours 30 mins)

8.4 评分标准

- 助教现场检查: 40 分
 - 向助教清楚解释设计要点、关键代码 20 分
 - 现场编译并成功运行代码 20 分
- 实验报告和用户使用手册: 30 分
 - 实验报告 20 分
 - 用户使用手册 10 分
- 源代码: 30 分
 - 主要数据结构、函数注释清楚 15 分
 - 代码结构清晰、可读性好 15 分